

Learning-based Problem Reduction for Large-scale Uncapacitated Facility Location Problems

Shuaixiang Zhang¹, Yixuan Yang¹, Hao Tong¹, Xin Yao^{2,1}

¹Guangdong Provincial Key Laboratory of Brain-inspired Intelligent Computation, Department of Computer Science and Engineering, Southern University of Science and Technology, Shenzhen, China

²Department of Computing and Decision Sciences, Lingnan University, Hong Kong SAR, China

Emails: {12010819, 12011316}@mail.sustech.edu.cn, htong6@outlook.com, xinyao@ln.edu.hk

Abstract—The uncapacitated facility location problem (UFLP) is a well-known combinatorial optimization problem, attracting numerous heuristic and meta-heuristic methods. However, these effective algorithms still encounter challenges when solving large-scale UFLP instances. To deal with the thousands of decision variables, one possible approach is to predict and remove unimportant facilities and thus decreases the dimensionality of problem instances. This paper explores the problem reduction of large-scale instances of UFLP. We propose suitable features of facilities to construct machine learning models for UFLP by learning from small instances. Based on this model, we introduce a novel problem reduction strategy-based optimization framework, and apply it to large instances. Through comprehensive experiments, we show that the proposed problem reduction strategy can effectively transform the UFLP instances to smaller ones. The performance of existing solution methods can be significantly enhanced, especially on large-scale instances. Under the proposed framework, we also show the generalization abilities of our models, which can be improved further.

Index Terms—Uncapacitated Facility Location Problem, Problem Reduction, Learning-based Optimization, Heuristic Algorithms, Meta-heuristic Algorithms

I. INTRODUCTION

The Uncapacitated Facility Location Problem (UFLP) is a well-known combinatorial optimization problem, which has a wide range of applications in logistics and supply chain management [1]. Given a set of candidate facility locations, the objective of UFLP is to choose a subset for facility establishment, aiming to efficiently serve all customers. The goal is to minimize the overall cost, including both the initial facility setup costs and the subsequent transportation costs incurred between facilities and customers.

Numerous studies have been conducted to address the UFLP, including exact algorithms and meta-heuristic approaches. Exact algorithms are able to guarantee to find the global optimum, such as the branch-and-bound algorithms [2], [3] and Lagrangian relaxation methods [4], [5], etc. However, these algorithms demand substantial computational resources, especially for large-scale optimization problems since UFLP is NP-hard. Therefore, meta-heuristic methods are needed for tackling large UFLP. Basu et al. [6] analyzed four meta-heuristics for discrete facility location problems including genetic algorithms, tabu search, particle swarm optimization,

and scatter search. These meta-heuristic algorithms can search high-quality solutions quickly, but their effectiveness still diminishes as the sizes of UFLP instances increase [6].

Large-scale combinatorial optimization problems are usually encountered in real-world scenarios [7], often featuring thousands of decision variables. The high dimension poses challenges even for meta-heuristic algorithms, leading to impractical computational time and resource requirements. A promising approach to deal with this issue [8] is to identify and eliminate unimportant decision variables and thereby reducing problem instance size and facilitating optimization.

Recently, problem reduction using machine learning has attracted more attention [8], [9]. Sun et al. [8] introduced a novel approach utilizing direct graph features and solution space features for learning-based problem reduction, specifically predicting and removing unimportant nodes in the graph of the maximum weight clique problem (MWCP). Building upon this, in their subsequent work [9], they applied a learning-based problem reduction strategy to the traveling salesman problem (TSP). Here, the difference is to predict and remove unimportant edges from the graph rather than the nodes. In addition, instead of finding appropriate features to leverage classic machine learning algorithms, Arciprete et al. [10] employed Graph Neural Network layers to find a reduction strategy. And they performed pruning for the Maximum Clique Enumeration problem to obtain a reduced network.

The existing literature on problem reduction driven by machine learning focused on problems with common graphs, such as the TSP. They proposed some features of each node or edge in the graph as the features of decision variables. However, the graph of UFLP is very different from normal graphs considered in the existing literature. It is a complete bipartite graph, where each edge connects a facility and a customer. Rather than selecting some nodes in MWCP [8] or some edges in TSP [9], the UFLP is an assignment problem that assigns customers to appropriate facilities. Consequently, common graph features proposed in existing literature such as degrees become ineffective in our scenario. This paper aims to design appropriate features for UFLP and apply problem reduction techniques to large-scale instances, to address the challenges of the high dimension by learning from small instances. The two major contributions of this paper are as

Corresponding author: Xin Yao (xinyao@ln.edu.hk).

follows:

- The appropriate features are designed for the facility of UFLP, and their relationships with the reduction probabilities are learned from small instances.
- The problem reduction strategy-based optimization framework is proposed to transform a large-scale instance into a smaller one for efficient optimization. The experimental studies demonstrate that the framework is effective to transform the problem instances and enhance optimization of large problem instances.

The remainder of the paper is organized as follows. Section II provides the formulation of the UFLP and introduces related work about the learning-based problem reduction. In Section III, the framework for applying the problem reduction method to solve the UFLP is proposed. Section IV presents the experimental results and analysis. Finally, the conclusion and future work are provided in Section V.

II. BACKGROUND

A. Problem Formulation of UFLP

Generally, considering a set of customers I and candidate facilities J that each customer $i \in I$ is required to be served by one of the facilities $j \in J$ with a serving cost c_{ij} and each facility j has an opening cost f_j , UFLP aims at choosing a subset of candidate facilities to be opened and assigning each customer to one facility with smallest total costs including serving costs and opening costs. The assigning condition between customer i and facility j is represented as y_{ij} . Suppose it has m candidate facilities and n customers, the UFLP instance can be represented by $P = (I, J, F, C)$, and a solution of UFLP can be represented by facility decision variables $X = [x_1, x_2, \dots, x_m]$:

$$x_j = \begin{cases} 1, & \text{if facility } j \text{ is opened} \\ 0, & \text{otherwise} \end{cases}, \quad (1)$$

Therefore, the objective function and constraints for UFLP are given as follows [1]:

$$\text{Minimize}_X \left\{ \sum_{j \in J} f_j x_j + \sum_{i \in I} \sum_{j \in J} c_{ij} y_{ij} \right\}, \quad (2)$$

subject to:

$$\sum_{j \in J} y_{ij} = 1, \forall i \in I, \quad (3)$$

$$0 \leq y_{ij} \leq x_j, \forall i \in I, \forall j \in J, \quad (4)$$

The objective of UFLP in Eq. (2) is to minimize the total cost of two parts, one is the opening cost of facilities and the other is the service cost between facilities and customers. For each facility, it can only be opened or closed according to Constraint 1. And for each customer, Constraint 3 guarantees that it must be served by only one facility. Additionally, Constraint 4 makes sure that closed facilities can not serve any customers.

B. Learning-based Problem Reduction Methods

The learning-based problem reduction methods typically involve the prediction and removal of unimportant decision variables to reduce the dimensionality of the problem. The general framework of this approach can be outlined as follows:

- 1) Calculate the features of the decision variables;
- 2) Input the calculated features into the pre-trained model;
- 3) Obtain the model outputs, representing a form of probability likelihood;
- 4) Reduce the decision variables based on the outputs of the model.

In the above framework, the pre-trained model can be obtained by training a model using data collected from small problem instances where the optimal solutions can be easily obtained. Ensuring accurate predictions is necessary for achieving the effective reduction. Therefore, the identification of appropriate features is critical in the framework. Following accurate predictions, Step 4 above decreases instance sizes by eliminating decision variables with unfavorable outcomes.

In order to construct appropriate features for decision variables, various reduction methods regard the decision variables as nodes or edges of graphs have been proposed. Lauri et al. [11] employed common graph features, such as vertex degree, to predict and remove unimportant decision variables. Additionally, they leveraged Pearson's chi-square statistic for enhanced feature capturing. Building on this methodology, Grassia and Lauri et al. [12], [13] proposed a multi-stage learning approach to prune the search space. What's more, Fitzpatrick et al. [14] identified some features derived from linear programming relaxation, cutting planes exploration, minimum-weight spanning tree heuristics and various other local and statistical analysis of the graph. Based on these features, they proposed a pruning heuristic to get sparse networks by leveraging machine learning.

Besides using the graph features, Sun et al. [8], [9] also computed two kinds of features from stochastic sampling solutions based on rank and correlation, providing more accurate prediction and better performance for the problem reduction. What's more, instead of finding appropriate features to leverage classic machine learning algorithms, Arciprete et al. [10] employed Graph Neural Network (GNN) layers to find a reduction strategy. They use geometric deep learning models with graph attention network layers to learn the node embedding, which is then classified by a multi-layer perceptron. And they apply it to perform a single pruning stage for Maximum Clique Enumeration (MCE) problem to get a reduced network.

However, the UFLP is actually an assigning problem which is very different from normal graph-based problems in the existing literature. Although it can be regarded as a complete bipartite graph where edges only exist between facilities and customer nodes, the edges connect totally different types of nodes whereas all nodes are in the same type in the normal graph. Consequently, common graph features such as degrees become ineffective in the UFLP.

Tayebi et al. [15] focus on the k-median problems and leverage solutions of the LP-relaxation problem to design suitable features. However, it is still time-consuming to obtain these solutions on large-scale instances with more than 1000 facilities. In addition, their features need to use the distances between facilities, which are not available on some benchmarks, such as M* [16] and Gap [17]. Therefore, we need to construct better and more efficient features for facilities in UFLP to leverage learning-based problem reduction.

III. PROBLEM REDUCTION STRATEGY-BASED OPTIMIZATION FRAMEWORK

We have designed a Problem Reduction Strategy-based Optimization Framework for the UFLP. This approach involves learning the characteristics of different facilities on a multitude of small instances to create a pre-trained model. This model is then utilized in large-scale instances to calculate the opening probability of each facility in a very short time. Facilities with a probability below a certain threshold are eliminated, thereby achieving a reduction in large-scale instances and significantly decreasing the problem's dimension. Then, the transformed instance with a smaller dimension can be solved by an existing meta-heuristic algorithm to find high-quality solutions in a shorter time.

Furthermore, we need a robust metric to determine which facilities are worth opening and which are not. In the UFLP, our goal is to open a subset of facilities and serve all customers with the smallest possible total costs which includes the opening cost of facilities and the service costs. Therefore, in Section III-B, we design the **Cost Matrix** considering these two types of costs, and then derive the **Rank Matrix**, which provides the efficiency of each facility in a comparative perspective. And in Section III-C, we introduce how to extract the related characteristics of each facility from the **Rank Matrix**, and create a feature vector to serve as the input of the model. In Section III-D, we introduce the design of our reduction policy for the UFLP.

A. System Overview

Our system's primary architecture is presented in Figure 1, which leverages a model trained on small instances and applies it to larger cases, thereby aiding in the reduction of decision variables for these larger instances. We initially generated N_{train} small instances randomly (the specific generation method can be found in Section IV-A1). Then, following our features designed in Section III-C, we assign a vector containing information in four dimensions to each facility, which served as the input for our model meanwhile does not consume much time.

Since obtaining the optimal solution in small instances is relatively easy, we utilize the Cplex solver [18] to obtain optimal solutions in these smaller instances. For each facility in an UFLP instance, if the optimal solution involved opening this facility, we marked its label as '1', and otherwise '0'. This served as the output for our model. Thus, according to Algorithm 2, we train a learning model that, given the features

of a facility in an instance as input, outputs the opening probability for the corresponding facility.

Similarly, as computing the feature vector for an instance does not consume excessive time, we calculate the feature vector for each facility in large instances and input these into our pre-trained model. According to the output of the model, facilities which seem irrelevant to the optimal solution are removed from the instance, resulting in a reduced and smaller-scale instance. Then, the transformed instances are solved by an existing meta-heuristic algorithm to find high-quality solutions quickly. The final framework for solving large-scale UFLP is shown in Algorithm 1.

B. Cost Matrix and Rank Matrix

Intuitively, a facility with a lower opening cost and lower service costs to most customers it serves is more likely to be open. In order to provide a comprehensive view of these two types of costs, we have designed the **Cost Matrix** and **Rank Matrix**, further generating a Feature Vector for each facility with four features in Section III-C. An illustration of the **Cost Matrix** and the **Rank Matrix** is presented in Figure 2.

1) **Cost Matrix**: Considering both the impact of a facility's opening cost and service costs, the **Cost Matrix** is designed, in which the vertical axis represents each facility and the horizontal axis represents each customer. For the facility j and the customer i , the corresponding cell in the **Cost Matrix** is the product of the corresponding service cost c_{ij} and the opening cost of that facility f_j , denoted as $f_j * c_{ij}$.

In the cost matrix, the lower f_j and the lower c_{ij} mean a better connection between the facility and the customer. In addition, the opening cost of a facility f_j is significantly higher than the service cost c_{ij} in most problem instances, which means the smaller c_{ij} would be easily diluted by the much larger f_j . By using $f_j * c_{ij}$, the impact of both values can be equally represented, providing a more balanced and realistic assessment.

2) **Rank Matrix**: Once we have the **Cost Matrix**, our goal is to extract the efficient characteristics of each facility from the information it contains. For each customer, we sort values filled in all cells corresponding to this customer across different facilities. We then replace the original values in each cell with the rank of that facility for that particular customer. This process effectively indicates the relative performance of each facility in comparison with others for a specific customer.

However, there is an issue that as the size of the instance varies, the distribution of our ranks changes accordingly. Since the rank can range from 1 to m (where m is the number of facilities), the rank values for facilities are generally lower in smaller problem instances, while they tend to be higher in larger problem instances. Therefore, it becomes necessary to perform the normalization to eliminate the effect of instance size on the distribution of the rank value. This is simply achieved by dividing the rank by m . It ensures that the final values of the ranking matrix lie between $\frac{1}{m}$ and 1.

The rank matrix is finally obtained by applying ranking and normalization to all columns. Each cell contains a rank r_{ij}

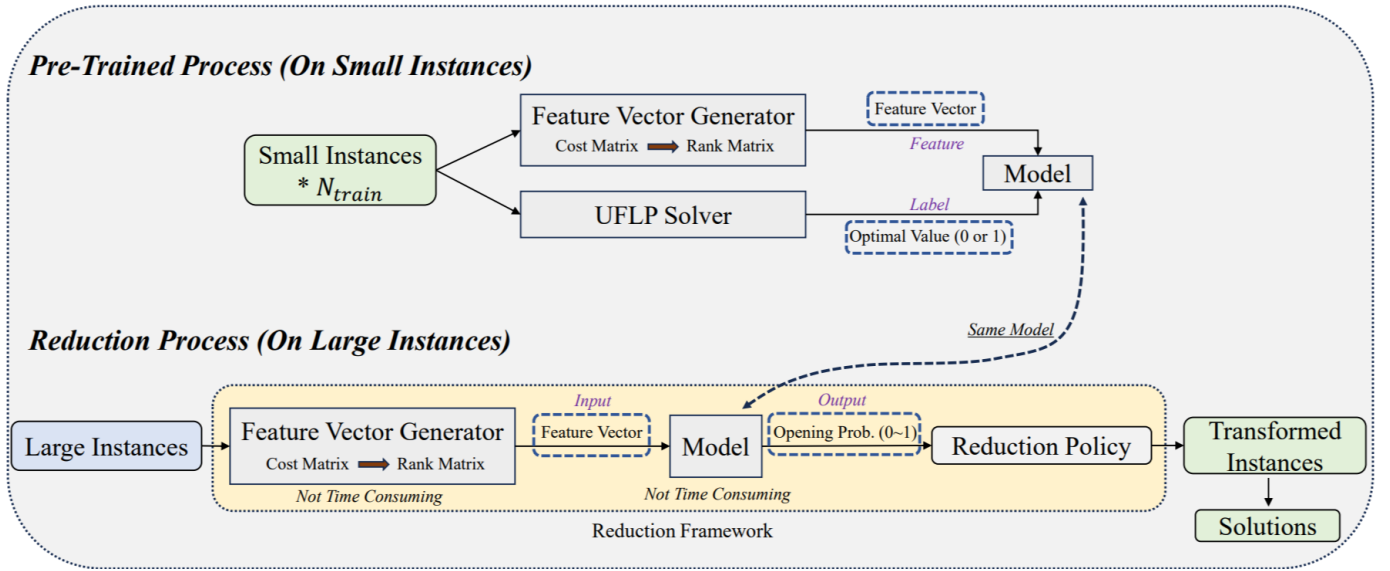


Fig. 1. System Overview of the Problem Reduction Strategy-Based Optimization Framework

Algorithm 1 Problem Reduction Strategy-based Optimization Framework for UFLP

Input: The problem instance $P = (I, J, F, C)$; The pre-trained model $Model$.

Output: The best solution $best$.

- 1: Initialize empty set of deleted facilities $S = \emptyset$.
- 2: $M_{rank} \leftarrow$ Calculate rank matrix according to F, C .
- 3: **for** each facility $j \in J$ **do**
- 4: $feature \leftarrow$ Calculate features of j from M_{rank} .
- 5: $output \leftarrow Model(feature)$.
- 6: **if** $output == 0$ **then**
- 7: $S \leftarrow S \cup \{j\}$.
- 8: **end if**
- 9: **end for**
- 10: $J' \leftarrow J \setminus S$.
- 11: $P' = (I, J', F, C)$.
- 12: $best \leftarrow$ Solve P' using the meta-heuristic algorithm.
- 13: **return** $best$

that represents the economic competitiveness of a facility in relation to a specific customer, in the context of competing with other facilities. The rank matrix, therefore, provides a clear comparative perspective on the efficiency and appeal of each facility in the context of each customer.

C. Feature Vector Design

After obtaining the rank matrix, there is a basis for evaluating whether a facility is worth opening for all customers. If a facility has relatively low rank values for all customers, we consider it overall worth opening.

Therefore, for each facility, we calculate the mean, standard deviation, skewness, and kurtosis of all its rankings across all

customers. Among them, the skewness describes the asymmetry degree of the data, which is a measure of the skew direction and degree of data distribution. And the kurtosis intuitively represents the peak height of the probability distribution curve at the average. The conventional skewness and kurtosis are as follows [19], where $\mu = E(r_{ij})$ and $\sigma^2 = E(r_{ij} - \mu)^2$.

$$Skew_j = E\left(\frac{r_{ij} - \mu}{\sigma}\right)^3, \quad (5)$$

$$Kurt_j = E\left(\frac{r_{ij} - \mu}{\sigma}\right)^4 - 3. \quad (6)$$

These four statistical measures are combined into a four-dimensional vector, which then serves as the input of our model. This approach ensures that each facility's performance is thoroughly evaluated, providing a comprehensive and detailed understanding of its potential impact.

D. Reduction Policy

In this part, we simply train the model with a binary classification task to leverage the output of the model to decide the reduction policy. When the output is 1, we consider the facility worth opening and thus retain it; when it is 0, we deem the facility less viable and remove it from the decision variables of the original problem. By doing this, we effectively reduce the scale of the problem.

IV. EXPERIMENTAL STUDIES

The primary objective of our experiments is to demonstrate the efficacy of our reduction strategy and framework. Firstly, we measure the effectiveness of the transformed instances, considering their global optima after using the problem reduction strategy. Secondly, the optimization algorithm is applied to both the original and transformed instances to validate whether

Cost Matrix

	Customer 1	Customer 2	Customer 3		Customer i		Customer n
Facility 1	$f_1 * c_{11}$	$f_1 * c_{21}$	$f_1 * c_{31}$	...	$f_1 * c_{i1}$	...	$f_1 * c_{n1}$
Facility 2	$f_2 * c_{12}$	$f_2 * c_{22}$	$f_2 * c_{32}$	...	$f_2 * c_{i2}$	...	$f_2 * c_{n2}$
.....	...	...	...	...	...	...	...
Facility j	$f_j * c_{1j}$	$f_j * c_{2j}$	$f_j * c_{3j}$	...	$f_j * c_{ij}$	...	$f_j * c_{nj}$
.....	...	...	...	...	...	...	...
Facility m	$f_m * c_{1m}$	$f_m * c_{2m}$	$f_m * c_{3m}$	...	$f_m * c_{im}$	...	$f_m * c_{nm}$

Get Rank for each Customer (Column) across all Facilities

Rank Matrix

	Customer 1	Customer 2	Customer 3		Customer i		Customer n
Facility 1	r_{11}/m	r_{21}/m	r_{31}/m	...	r_{i1}/m	...	r_{n1}/m
Facility 2	r_{12}/m	r_{22}/m	r_{32}/m	...	r_{i2}/m	...	r_{n2}/m
.....	...	...	...	...	...	...	...
Facility j	r_{1j}/m	r_{2j}/m	r_{3j}/m	...	r_{ij}/m	...	r_{nj}/m
.....	...	...	...	...	...	...	...
Facility m	r_{1m}/m	r_{2m}/m	r_{3m}/m	...	r_{im}/m	...	r_{nm}/m

Fig. 2. Cost Matrix and Rank Matrix: For an instance, first construct its Cost Matrix, and then derive its Rank Matrix based on the Cost Matrix.

Algorithm 2 Pseudo-code for training model

Input: The generated problem instances PS .

Output: The pre-trained model $Model$.

```

1: Initialize empty set of training data  $D \leftarrow \emptyset$ .
2: for each generated instance  $P = (I, J, F, C)$  in  $PS$  do
3:    $M_{rank} \leftarrow$  Calculate rank matrix according to  $F, C$ .
4:    $X \leftarrow$  The optimal solution of solving  $P$ .
5:   for each facility  $j$  from  $J$  do
6:      $feature \leftarrow$  Calculate features of  $j$  from  $M_{rank}$ .
7:      $label \leftarrow x_j$ .
8:      $D \leftarrow D \cup \{(feature, label)\}$ .
9:   end for
10: end for
11:  $Model \leftarrow$  Train model on  $D$ .
12: return  $Model$ 

```

the problem reduction strategy-based optimization framework enhances the performance of existing algorithms. Finally, the generalization ability of our algorithm is evaluated across various benchmarks, including random instances, the M^* [16], Gap [17], and a part of OR [20] benchmarks.

A. Experimental Settings

1) **Model Training:** To obtain sufficient training data, we will generate numerous small UFLP instances where global optima are known. The following instance generators will be utilized:

1) **Random generator:** An UFLP instance is generated by randomly selecting n and m points as the customers and candidate facilities in a square with a ratio of $n/m \in [1, 10]$. The service costs are the Euclidean distances between the facilities and the customers, which satisfies $c_{ij} \in [1, 200]$ by fixing the width of the square as 141. And the opening cost of each facility f_j is set as $f_j \in [10, 200\alpha]$, and α ranges from 1 to 20 randomly, which can decide the proportional relationship of f_j and c_{ij} .

2) **OR generator** [20]: Besides the instances collected from real cases and their variants, OR also contains three large instances generated by Beasley [20]. This

generator randomly selects some points in a square as the customers and candidate facilities, and uses the Euclidean distance between them to design service costs. Then, opening costs for facilities are designed according to a defined expected number p of opening facilities.

- 3) **M^* generator** [16]: It generates the customer demands randomly from $[b_{min}, b_{max}]$ and the service costs randomly from $[c_{min}, c_{max}]$ considering the customer demands. Then it designs opening costs for facilities from $[f_{min}, f_{max}]$, considering that the facility with lower service costs often has a higher opening cost.
- 4) **Gap generator** [17]: There are many infinite service costs in the Gap data set. Each customer has exactly l valid service costs in GapA, and each facility has exactly l valid service costs in GapB. In GapC, each customer has exactly l valid service costs while each facility has exactly l valid service costs.

In order to assess the efficacy of transformed problem instances and the performance enhancement of optimization algorithms, we generate small UFLP instances with $m \in [5, 30]$ using the Random generator for model training. After that, to evaluate the generalization ability of the proposed framework, we employed Random, OR , M^* , and Gap generators to generate small training instances. The small instances generated by Random and M^* generators have $m \in [5, 30]$ facilities, and the detailed parameter settings of M^* generator are in Table I. Since OR generator expect no less than 2 opening facilities and Gap only has 10% valid service costs, it makes little sense for them to generate small instances with few facilities. Therefore, we only generate small instances with $m \in [20, 30]$ using OR and Gap generator. And p is randomly selected from $\{2, 3, 4, 5\}$ and l is randomly selected from $\{2, 3, 4\}$.

TABLE I
THE PARAMETER SETTINGS OF M^* GENERATOR

	m	n	f_{min}	f_{max}	c_{min}	c_{max}	b_{min}	b_{max}
<i>train</i>	5~30	$m \sim 10 * m$	$m/2$	$m*3$	0.2	1	1	5
<i>MS</i>	1000	1000	200	1200	0.5	5	1	5
<i>MT</i>	2000	2000	400	2400	0.5	5	1	5
<i>MU</i>	3000	3000	600	3600	0.5	5	1	5
<i>MV</i>	5000	5000	400	2400	0.125	2.5	1	5
<i>MW</i>	10000	10000	800	4800	0.125	2.5	1	5

We train the model on $N_{train} = 3000$ generated small

instances. For Gap benchmark, there are three types of instances: GapA, GapB and GapC. Therefore, we generate 1000 instances for each type of them to get 3000 Gap instances. All of the labels are determined by the optimal values solved using CPLEX solver [18], marking a facility as ‘1’ if it is open, and ‘0’ if not. We use the XGBoost [21] to train our model.

Although it has the same labels as the solution prediction task, the problem reduction task is different, and it wants to preserve decision variables which are relevant to the optimal solution as much as possible. Aiming at this, it is acceptable to keep more decision variables, compared with the solution prediction task. Therefore, the weight of data with the label ‘1’ should be higher in the problem reduction task. In addition, during the training process, an imbalanced classification is met since the facilities labeled as ‘1’ constitute only a small portion in most instances. Therefore, an oversampling strategy is employed before training.

For each dataset, we generate instances and train models for 25 independent runs, and retain $\frac{1}{10}$ data for testing when training the model. For each category, the precision, recall and F1 score of the model are recorded and shown in Table II. According to the results, the model performs well on the Random and M* datasets, but performs worse on OR and Gap datasets.

TABLE II
THE PRECISION, RECALL, F1 SCORE OF THE MODEL ON TRAINING INSTANCES

	precision		recall		F1 score	
	0	1	0	1	0	1
Random	0.92±0.01	0.73±0.01	0.93±0.0	0.68±0.01	0.92±0.0	0.7±0.01
M*	0.96±0.0	0.78±0.02	0.96±0.0	0.75±0.02	0.96±0.0	0.76±0.01
OR	0.89±0.0	0.45±0.01	0.93±0.0	0.35±0.01	0.91±0.0	0.39±0.01
Gap	0.71±0.01	0.53±0.01	0.77±0.01	0.44±0.01	0.74±0.01	0.48±0.01

2) *Optimization Algorithm*: We employed the discrete particle swarm optimization algorithm (DPSO) [22] as our testing algorithm to solve the UFLP. The population size, the maximum number of fitness evaluations of the DPSO are presented in Table III. And all other parameters of the DPSO are set the same as the original paper [22].

TABLE III
THE SETTINGS OF POPULATION SIZES AND TERMINATION CONDITIONS

	Size	Population size	Fitness evaluation number
MO	100	20	1800
MP	200	20	2500
MQ	300	20	3500
MR	500	20	4000
MS	1000	30	7000
MT	2000	50	15000
MU	3000	70	22000
MV	5000	90	26000
MW	10000	100	51000

All experiments are implemented in Python 3.7 and run on Intel(R) Core(TM) CPU i5-1240P 1.70GHz.

B. The Effectiveness of Problem Reduction Strategy

To measure the effectiveness of the problem reduction strategy used in the framework, the following criteria were defined:

- **Essential Proportion**: It is the minimum proportion of facilities required for the instance to be reduced without losing the original optimal solution.
- **Optimum Discrepancy**: It measures the difference between the global optima of the transformed problem instance and the corresponding original instance.
- **Retention Ratio**: It denotes the proportion of remaining facilities in the transformed problem instance.

The experimental results over 25 independent runs are presented in Table IV. From the results, we can observe that the average essential proportion on M* is only about 11.6% and our strategy can reduce the size of the instances to 13.2%. In addition, the transformation has no significant influence on the optimal solution for the given problem instance. Since it is difficult to find high-quality solutions which are better than the optima of the transformed instances for large-scale instances, it does not matter a lot when doing problem reduction on large UFLP instances.

TABLE IV
EXPERIMENTAL RESULTS ABOUT THE EFFECTIVENESS OF PROBLEM REDUCTION STRATEGY ON M*

Instances	Size	Essential Proportion	Retention Ratio	Optimum Discrepancy
MO1	100	0.093 ± 0.033	0.168 ± 0.023	0.0 ± 0.0%
MO2	100	0.112 ± 0.052	0.149 ± 0.017	0.156 ± 0.432%
MO3	100	0.302 ± 0.141	0.134 ± 0.021	0.597 ± 0.638%
MO4	100	0.114 ± 0.051	0.155 ± 0.019	0.669 ± 1.193%
MO5	100	0.304 ± 0.137	0.144 ± 0.033	0.351 ± 0.259%
MP1	200	0.118 ± 0.056	0.166 ± 0.027	0.065 ± 0.182%
MP2	200	0.081 ± 0.067	0.128 ± 0.028	0.121 ± 0.33%
MP3	200	0.062 ± 0.048	0.128 ± 0.026	0.054 ± 0.263%
MP4	200	0.098 ± 0.072	0.138 ± 0.023	0.016 ± 0.043%
MP5	200	0.116 ± 0.061	0.135 ± 0.021	0.051 ± 0.086%
MQ1	300	0.056 ± 0.038	0.133 ± 0.026	0.116 ± 0.477%
MQ2	300	0.073 ± 0.051	0.143 ± 0.022	0.05 ± 0.188%
MQ3	300	0.054 ± 0.024	0.123 ± 0.03	0.0 ± 0.0%
MQ4	300	0.053 ± 0.019	0.136 ± 0.023	0.0 ± 0.0%
MQ5	300	0.109 ± 0.052	0.152 ± 0.021	0.085 ± 0.187%
MR1	500	0.098 ± 0.093	0.108 ± 0.019	0.03 ± 0.087%
MR2	500	0.14 ± 0.113	0.089 ± 0.019	1.167 ± 0.999%
MR3	500	0.134 ± 0.095	0.091 ± 0.023	0.317 ± 0.419%
MR4	500	0.119 ± 0.072	0.102 ± 0.018	0.499 ± 0.678%
MR5	500	0.085 ± 0.052	0.116 ± 0.019	0.095 ± 0.186%

TABLE V
EXPERIMENTAL RESULTS OF BOOSTING THE PERFORMANCE OF DPSO ON LARGE M* INSTANCES

Instances	Size	Retention Ratio	$Best_O$	$Best_T$
MS1	1000	0.09±0.024	6207.521±470.959	5294.729±16.265
MS2	1000	0.082±0.023	6324.864±535.887	5287.353±17.361
MS3	1000	0.101±0.024	6149.201±549.374	5078.651±11.612
MS4	1000	0.085±0.022	6339.821±519.191	5442.681±18.164
MS5	1000	0.1±0.021	6146.088±633.019	5081.816±18.113
MT1	2000	0.088±0.022	11780.447±912.407	10123.406±34.72
MT2	2000	0.099±0.022	11265.777±1116.53	9512.996±32.31
MT3	2000	0.075±0.024	12775.554±754.363	10974.832±76.294
MT4	2000	0.078±0.023	12287.358±1068.697	10346.45±21.271
MT5	2000	0.089±0.024	11947.934±914.655	10157.144±26.499
MU1	3000	0.082±0.027	19312.859±1760.928	15590.024±82.821
MU2	3000	0.079±0.027	19084.218±1941.494	15535.922±59.37
MU3	3000	0.087±0.025	18458.809±1631.283	15143.267±58.445
MU4	3000	0.081±0.026	19123.324±1578.516	16131.371±48.128
MU5	3000	0.084±0.024	18025.03±1491.425	15071.128±78.93
MV1	5000	0.081±0.024	26077.568±5123.883	9813.072±20.315
MW1	10000	0.057±0.028	103857.279±15693.471	20700.817±173.998

TABLE VI
EXPERIMENTAL RESULTS ABOUT GENERALIZATION ABILITIES OF THE REDUCTION ALGORITHM

Instances	Random		OR		M*		Gap	
	$Best_O$	$Best_T$	$Best_O$	$Best_T$	$Best_O$	$Best_T$	$Best_O$	$Best_T$
Random0	-	-	13990.71	16957.89	18568.2	15243.86	17764.22	19323.54
Random1	-	-	18581.05	19705.25	8651.04	13506.78	8658.77	9959.97
Random2	-	-	14691.98	17398.3	9950.38	12071.72	9894.87	11997.51
OR0	11747389.78	11211311.45	-	-	11760719.91	-	11747654.01	11176411.91
OR1	13238970.82	12212621.23	-	-	13116102.0	-	13211187.01	12371169.48
OR2	4096040.58	5205380.78	-	-	4111929.66	-	4096972.77	4358547.43
M0	6013.21	5123.77	6066.55	5217.89	-	-	6124.31	5285.07
M1	5853.16	4887.07	5692.73	4953.37	-	-	5877.83	5081.39
M2	6430.53	5627.11	6443.13	5705.48	-	-	6443.77	5866.04
Gap0	111556.56	110179.56	112262.52	2049604.88	112855.12	177934.52	-	-
Gap1	113468.44	117508.32	113831.36	-	112271.8	222018.76	-	-
Gap2	113214.4	110167.64	111428.0	-	112150.04	482575.32	-	-

C. The Efficacy of Proposed Framework

To evaluate whether the proposed optimization framework promotes the performance of existing optimization algorithms, we applied DPSO [22] to both original instances and transformed instances. And the experiment was implemented on the MS1, MT1 of M* benchmark and 15 large instances with $m \in [1000, 10000]$ generated using the M* generator. The parameter settings of M* generator are in Table I. The mean and standard deviation of total costs of obtained best solution found on original problem instances (referred to as $Best_O$) and the transformed problem instances (referred to as $Best_T$) by DPSO over 25 independent runs are presented in Table V. The better results under the Wilcoxon signed-rank test with a significance level of 0.05 are highlighted in the table. It is clear that most of the performance of DPSO on transformed instances is significantly better than on original instances.

The convergence curves of DPSO on MS1, MT1 are presented in Figure 3, and that of MV1, MW1 are presented in Figure 4. DPSO converges quickly on transformed instances, especially in larger instances, while it requires substantial computational power to converge on original instances. Therefore, our optimization framework is able to accelerate the speed of convergence significantly on generated M* instances, especially on large-scale instances.

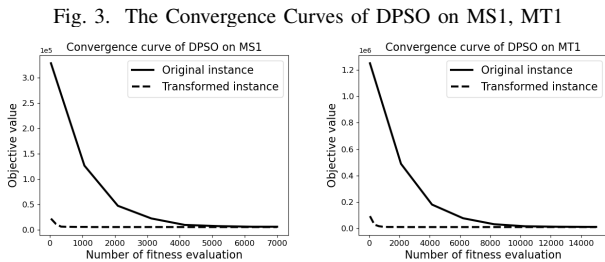
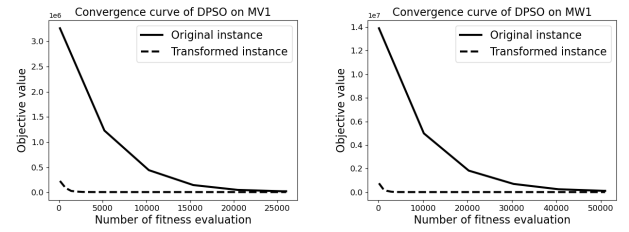


Fig. 3. The Convergence Curves of DPSO on MS1, MT1

D. The Generalization Ability

In order to evaluate the generalization ability of our algorithm, the cross-validation experiments are implemented on Random, OR, M*, Gap benchmarks. And 3 large testing

Fig. 4. The Convergence Curves of DPSO on MV1, MW1



instances with $m, n = 1000$ are generated using each generator. The OR0, OR1, OR2 are respectively generated by OR generator with p equal to 10, 20 and 30. And the parameter settings of M* generator are like MS in Table I. In addition, the Gap0, Gap1, Gap2 are respectively instances of type GapA, GapB and GapC, all with $l = 100$. Similar to the experiment in Section IV-C, we calculate the $Best_O$ and $Best_T$ of the testing instances using DPSO given the limited fitness evaluations.

The experimental results are presented in Table VI over 25 independent runs. The Wilcoxon signed-rank test with a significance level of 0.05 is applied to the results and the better one is highlighted. In the table, each row represents the testing instance and each column represents the training data set. In addition, the testing on the same training instances generator and testing instances generator is not implemented with result '-'. The remaining '-' means there are instances where all of the facilities are removed.

From the results, we can observe that models trained on random instances perform well on other instances, and models trained on all other instances perform well on M* instances. However, models trained on OR, M, Gap instances sometimes perform worse and affect the performance of DPSO. For example, the random instances are transformed worse using models trained on other benchmarks, which impairs the efficacy of DPSO significantly. And the model trained on M* can not boost the performance of DPSO on most of the other benchmarks.

For the Random instances, the different generation methods of opening cost cause a very different distribution compared

with other benchmarks, which can not be handled by the proposed feature. And in the Gap instances, there are 90 % extremely large service costs while the valid service costs are too small compared with the opening cost. In this condition, selecting the smallest number of facilities to serve all customers validly becomes the most critical issue. However, it is difficult for the proposed feature to identify the efficient combination of facilities.

One possible way for dealing with these issues is to introduce the similarity between facilities into features, since similar facilities are not likely to open together. In addition, it is also useful to consider appropriate combinations of facilities. In UFLP, each customer is served using the minimum service cost from opening facilities, and we want to reduce the sum of the service cost without opening too many facilities. Therefore, facilities with complementary service cost vectors are useful and easily be opened at the same time.

V. CONCLUSION

This paper introduces a new learning-based meta-heuristic algorithm for large-scale UFLPs. It first identifies essential features of the UFLP. Then small-size UFLPs are used as training data to train a model that can remove unimportant facilities. Next, such a pre-trained model is applied to large-scale UFLPs to reduce the problem dimension of large-scale UFLPs. Finally, a meta-heuristic algorithm, a PSO algorithm in this paper, is used to tackle the reduced problem instances. Extensive experimental studies have demonstrated the effectiveness and efficiency of our proposed methods for feature identification and optimization.

In particular, our experimental results demonstrate that our proposed reduction strategy can effectively transform the UFLP instances, reducing their sizes to only 13.2% of the original on the M* benchmark problem set. Additionally, the proposed framework significantly enhances the performance of DPSO and promotes it converging quickly, especially for large instances.

Although our proposed framework has a good performance on M*, its generalization abilities on other benchmarks are not good enough. Our future work could focus on improving the features of facilities for UFLP further to handle different benchmarks. In addition, it would be interesting to study other problem dimension reduction methods than just predicting the usefulness of facilities. Finally, it is worth exploring problem features that are not at the facility level, but at the entire problem level.

ACKNOWLEDGMENT

This work was supported by the National Key R&D Program of China (Grant No. 2023YFE0106300), the National Natural Science Foundation of China (Grant No. 62250710682), the Guangdong Provincial Key Laboratory (Grant No. 2020B121201001), and the Program for Guangdong Introducing Innovative and Entrepreneurial Teams (Grant No.2017ZT07X386).

REFERENCES

- [1] V. Verter, "Uncapacitated and capacitated facility location problems," *Foundations of location analysis*, pp. 25–37, 2011.
- [2] A. Klose, "A branch and bound algorithm for an uncapacitated facility location problem with a side constraint," *International Transactions in Operational Research*, vol. 5, no. 2, pp. 155–168, 1998.
- [3] B. Goldengorin, D. Ghosh, and G. Sierksma, "Branch and peg algorithms for the simple plant location problem," *Computers & Operations Research*, vol. 30, no. 7, pp. 967–981, 2003.
- [4] J. Barcelo, Å. Hallefjord, E. Fernandez, and K. Jörnsten, "Lagrangean relaxation and constraint generation procedures for capacitated plant location problems with single sourcing," *Operations-Research-Spektrum*, vol. 12, no. 2, pp. 79–88, 1990.
- [5] C. Beltrán-Royo, J.-P. Vial, and A. Alonso-Ayuso, "Semi-lagrangian relaxation applied to the uncapacitated facility location problem," *Computational Optimization and Applications*, vol. 51, no. 1, pp. 387–409, 2012.
- [6] S. Basu, M. Sharma, and P. S. Ghosh, "Metaheuristic applications on discrete facility location problems: a survey," *Opsearch*, vol. 52, pp. 530–561, 2015.
- [7] K. Anstreicher, N. Brixius, J.-P. Goux, and J. Linderoth, "Solving large quadratic assignment problems on computational grids," *Mathematical Programming*, vol. 91, pp. 563–588, 2002.
- [8] Y. Sun, X. Li, and A. Ernst, "Using statistical measures and machine learning for graph reduction to solve maximum weight clique problems," *IEEE transactions on pattern analysis and machine intelligence*, vol. 43, no. 5, pp. 1746–1760, 2019.
- [9] Y. Sun, A. Ernst, X. Li, and J. Weiner, "Generalization of machine learning for problem reduction: a case study on travelling salesman problems," *OR Spectrum*, vol. 43, pp. 607–633, 2021.
- [10] A. Arciprete, V. Carchiolo, D. Chiavetta, M. Grassia, M. Malgeri, and G. Mangioni, "Geometric deep learning graph pruning to speed-up the run-time of maximum clique enumeration algorithms," in *International Conference on Complex Networks and Their Applications*. Springer, 2022, pp. 415–425.
- [11] J. Lauri and S. Dutta, "Fine-grained search space classification for hard enumeration variants of subset problems," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 33, no. 01, 2019, pp. 2314–2321.
- [12] M. Grassia, J. Lauri, S. Dutta, and D. Ajwani, "Learning multi-stage sparsification for maximum clique enumeration," *arXiv preprint arXiv:1910.00517*, 2019.
- [13] J. Lauri, S. Dutta, M. Grassia, and D. Ajwani, "Learning fine-grained search space pruning and heuristics for combinatorial optimization," *Journal of Heuristics*, pp. 1–35, 2023.
- [14] J. Fitzpatrick, D. Ajwani, and P. Carroll, "Learning to sparsify travelling salesman problem instances," in *International Conference on Integration of Constraint Programming, Artificial Intelligence, and Operations Research*. Springer, 2021, pp. 410–426.
- [15] D. Tayebi, S. Ray, and D. Ajwani, "Learning to prune instances of k-median and related problems," in *2022 Proceedings of the Symposium on Algorithm Engineering and Experiments (ALENEX)*. SIAM, 2022, pp. 184–194.
- [16] J. Kratica, D. Tošić, V. Filipović, and I. Ljubić, "Solving the simple plant location problem by genetic algorithm," *RAIRO-Operations Research*, vol. 35, no. 1, pp. 127–142, 2001.
- [17] Y. Kochetov and D. Ivanenko, "Computationally difficult instances for the uncapacitated facility location problem," *Metaheuristics: Progress as real problem solvers*, pp. 351–367, 2005.
- [18] I. I. Cplex, "V12. 1: User's manual for cplex," *International Business Machines Corporation*, vol. 46, no. 53, p. 157, 2009.
- [19] T.-H. Kim and H. White, "On more robust estimation of skewness and kurtosis," *Finance Research Letters*, vol. 1, no. 1, pp. 56–73, 2004.
- [20] J. E. Beasley, "An algorithm for solving large capacitated warehouse location problems," *European Journal of Operational Research*, vol. 33, no. 3, pp. 314–325, 1988.
- [21] T. Chen and C. Guestrin, "Xgboost: A scalable tree boosting system," in *Proceedings of the 22nd acm sigkdd international conference on knowledge discovery and data mining*, 2016, pp. 785–794.
- [22] A. R. Guner and M. Sevkli, "A discrete particle swarm optimization algorithm for uncapacitated facility location problem," *Journal of Artificial Evolution and Applications*, vol. 2008, 2008.